

Complexidade

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

Complexidade de tempo

- Formas de determinar o tempo de execução de um algoritmo.
- Possível verificar empiricamente, executando o algoritmo com diversas entradas.
 - Entretanto este tempo irá depender de aspectos do hardware e do software do computador que estiver sendo utilizado.
 - Os resultados dependem da linguagem de programação e do compilador utilizado.
 - Resultados dependem da quantidade de memória utilizada.
 - Dependem do ambiente operacional: hardware + sistema operacional.

Complexidade de tempo

- É possível utilizar métodos analíticos para obter uma ordem de grandeza do tempo de execução, independente do computador sendo utilizado.
 - Usamos um modelo matemático baseado em um computador idealizado.
 - Assumimos situações nas quais tenhamos grandes quantidades de dados que tendem a crescer;
 - O conjunto de operações a serem executadas deve ser especificado, assim como o custo associado com a execução de cada operação;
 - Não são consideradas constantes aditivas ou multiplicativas.

Complexidade de tempo

- Tempo de execução deve ser expresso em função da entrada.
- Cada etapa executada pelo algoritmo denominamos de passos. Cada passo consiste na execução de um número fixo de operações básicas cujos tempos de operações são considerados constantes.
 - Operação básica de maior frequência é chamada de dominante.
 - Algoritmo de um passo tem execução constante.
- A expressão matemática de avaliação do tempo de execução de um algoritmo pode ser definida como sendo uma função que fornece o número de passos efetuados pelo algoritmo a partir de uma certa entrada.

Complexidade de tempo

- O custo de execução de um algoritmo é definido como uma função de custo, ou função de complexidade f , sendo que $f(n)$ é a medida do tempo necessário para executar o algoritmo para um problema de tamanho n .
 - Função de complexidade de tempo do algoritmo.
 - Também é possível medir a quantidade de memória necessária para executar o algoritmo para um problema de tamanho n , e chamamos neste caso a função f de função de complexidade de espaço.

Exemplo

- Algoritmo para obter o máximo em uma sequência de valores

```
max = v[0]
```

```
for i = (1,n)
```

```
    if max < v[i] then
```

```
        max = i
```

- O número de passos é igual ao número de execuções do bloco **for**, sendo igual a $|n|$, com $n > 1$.

Exemplo

- Ex. Algoritmo de inversão de uma sequência

for $i = (1, n/2)$

temp = $S[i]$

$S[i] = S[n-i+1]$

$S[n-i+1] = \text{temp}$

- O número de passos é igual ao número de execuções do bloco **for**, sendo igual a $\lfloor n/2 \rfloor$, com $n > 1$.

Exemplos

- Algoritmo – Soma de Matrizes

for i = (1,n)

for j = (1,n)

$c[i,j] = a[i,j] + b[i,j]$

- Cada passo corresponde a uma soma $a[i,j] + b[i,j]$.
- O número total de passos é o número total de somas, que é igual a
 - $n * n = n^2$.

Exemplos

- Algoritmo – Produto de Matrizes

for i = (1,n)

for j = (1,n)

c[i,j] = 0

for k = (1,n)

c[i,j] = c[i,j] + a[i,k] * b[k,j]

- Um passo corresponde ao produto $a[i,k] * b[k,j]$.
- O número total de passos é o número total de produtos, que é igual a
 - $n * n * n = n^3$.

Formalização da Complexidade de tempo

- Seja A um algoritmo, $E = \{E_1, \dots, E_m\}$ o conjunto de todas as entradas possíveis de A . Denote por t_i , o número de passos efetuados por A , quando a entrada for E_i .
- Temos:
 - Complexidade do pior caso = $\max_{E_i \in E} \{t_i\}$;
 - Maior tempo de execução sobre todas as entradas de tamanho n ;
 - Complexidade do melhor caso = $\min_{E_i \in E} \{t_i\}$;
 - Menor tempo de execução sobre todas as entradas de tamanho n ;
 - Complexidade do caso médio = $\sum_{1 \leq i \leq n} p_i t_i$, sendo p_i a probabilidade de ocorrência da entrada E_i .
 - É preciso supor uma distribuição de probabilidades sobre o conjunto de entradas de tamanho n .

Formalização da Complexidade de tempo

- Exemplo: procurar um registro pelo seu valor de chave sequencialmente.
 - Melhor caso – o valor de chave procurado é o primeiro registro – $f(n) = 1$;
 - Pior caso – o valor de chave procurado é o último registro – $f(n) = n$;
 - Caso médio – o registro procurado possui igual probabilidade de aparecer em qualquer posição – $f(n) = (n+1) / 2$.

Formalização da Complexidade de tempo

- A complexidade de pior caso é mais importante por definir um limite superior para o número de passos que o algoritmo pode efetuar, sendo por isto a mais utilizada.
- Em geral, chamamos de complexidade a complexidade de pior caso.
- As complexidades de melhor caso e de caso médio são pouco utilizadas.
 - A de caso médio exige o conhecimento da distribuição de probabilidade.

Notação O

- Na definição da complexidade, a função que descreve o número de passos para um algoritmo é simplificada, considerando que constantes aditivas e multiplicativas são desprezadas.
 - $3n \rightarrow n$;
 - $n + 3 \rightarrow n$;
 - $4n + 3 \rightarrow n$.

Notação O

- O foco será em valores assintóticos, portanto valores de menor grau são desprezados.
 - $n^2 + n \rightarrow n^2$;
 - $6n^3 + 4n - 9 \rightarrow n^3$.

Notação O

- Formalizando:
 - Sejam f , h funções reais positivas de variável inteira n . Diz-se que f é $O(h)$, escrevendo $f = O(h)$, quando existir uma função constante $c > 0$ e um valor inteiro n_0 , tal que
 - $n > n_0 \Rightarrow f(n) \leq c * h(n)$.
 - A função h atua como um limite superior para valores assintóticos da função f .

Notação O

- Exemplos:

- $f = n^2 - 1 \rightarrow f = O(n^2);$
- $f = n^2 - 1 \rightarrow f = O(n^3);$
- $f = 403 \rightarrow f = O(1);$
- $f = 5 + 2 \log n + 3 \log^2 n \rightarrow f = O(\log^2 n);$
- $f = 5 + 2 \log n + 3 \log^2 n \rightarrow f = O(n);$
- $f = 3n + 5 \log n + 2 \rightarrow f = O(n);$
- $f = 5 * 2^n + 5n^{10} \rightarrow f = O(2^n).$

Notação O

- Ex. Algoritmo de inversão de uma sequência

for $i = (1, n/2)$

temp = $S[i]$

$S[i] = S[n-i+1]$

$S[n-i+1] = \text{temp}$

- O número de passos se mantém constante com entradas de mesmo tamanho, sendo igual a $\lfloor n/2 \rfloor$, logo sua complexidade é igual a $O(n)$.

Notação O

- Algoritmo fatorial não recursivo:

$f = 1$

for $i = (2, n)$ do

$f = i * f$

return f

- O número de passos não altera com entradas de mesmo tamanho. Complexidade é igual ao número de multiplicações $i * f$, sendo igual a $O(n)$.

Notação O

- Algoritmo – Soma de Matrizes

for $i = (1,n)$ do

 for $j = (1,n)$ do

$c[i,j] = a[i,j] + b[i,j]$

- O número total de passos é o número total de somas, que é igual a $n * n = n^2$. A complexidade é igual a $O(n^2)$.

Notação O

- Algoritmo – Produto de Matrizes

for $i = (1,n)$ do

 for $j = 1,n)$ do

$c[i,j] = 0$

 for $k = (1,n)$ do

$c[i,j] = c[i,j] + a[i,k] * b[k,j]$

- O número total de passos é o número total de produtos, que é igual a $n * n * n = n^3$. A complexidade é igual a $O(n^3)$.

Notação O

- Para a complexidade de algoritmos recursivos, determina-se o número total de chamadas ao procedimento recursivo, e calcula-se a complexidade de uma única chamada.

Função `Fat(int n): int`

`if n = 0 then`

`return 1;`

`else`

`return n * Fat(n-1)`

- A complexidade de uma chamada é constante e igual a $O(1)$. São executadas n chamadas à função `Fat`. Complexidade é igual a $O(n)$.

Notação O

- Operações com a notação O :
 - $O(g+h) = O(g) + O(h)$
 - $O(k * g) = k * O(g) = O(g)$

Notação O

- Dizemos que os problemas no quais as soluções algorítmicas mais eficientes possuem uma mesma função de complexidade pertencem à mesma classe.
- $f(n) = O(1)$ – Algoritmos de complexidade $O(1)$ são ditos de complexidade constante. O uso do algoritmo independe do tamanho de n . As instruções do algoritmo são executadas um número fixo de vezes.
- $f(n) = O(\log n)$ – um algoritmo de complexidade $O(\log n)$ é dito ter complexidade logarítmica. Este tempo de execução ocorre tipicamente em algoritmos que resolvem um problema transformando-o em problemas menores.

Notação O

- $f(n) = O(n)$ – Um algoritmo de complexidade $O(n)$ é dito ter complexidade linear. Pouco é realizado sobre cada elemento de entrada.
- $f(n) = O(n \log n)$ – Este tempo de execução ocorre tipicamente em algoritmos que resolvem um problema quebrando-o em problemas menores, resolvendo cada um deles independentemente e depois juntando as soluções.
- $f(n) = O(n^2)$ – Um algoritmo de complexidade $O(n^2)$ é dito ter complexidade quadrática. Algoritmos desta ordem de complexidade ocorrem quando os itens de dados são processados aos pares, muitas vezes em um anel dentro de outro.

Notação O

- $f(n) = O(n^3)$ – Um algoritmo de complexidade $O(n^3)$ é dito ter complexidade cúbica. Algoritmos desta ordem de complexidade são úteis apenas para resolver pequenos problemas.
- $f(n) = O(2^n)$ – Um algoritmo de complexidade $O(2^n)$ é dito ter complexidade exponencial. Algoritmos desta ordem de complexidade geralmente não são úteis sob o ponto de vista prático.

Notação O

- Um algoritmo cuja função de complexidade é $O(c^n)$, $c > 1$, é chamado de algoritmo exponencial no tempo de execução.
- Um algoritmo cuja função de complexidade é $O(p(n))$, onde $p(n)$ é um polinômio, é chamado de algoritmo polinomial no tempo de execução.
- Distinção significativa quando o tamanho do problema cresce.
- Um problema é considerado intratável se ele é tão difícil que não existe um algoritmo polinomial para resolvê-lo.
- Um problema é considerado bem resolvido quando existe um algoritmo polinomial para resolvê-lo.

Exemplos

- Problema da pesquisa em lista de elementos desordenados.
 - Solução – busca sequencial – complexidade linear.
- Problema da pesquisa em lista de elementos ordenados.
 - Melhor solução – busca binária – complexidade logarítmica.
- Problema da soma de matrizes.
 - Complexidade quadrática.
- Problema da multiplicação de matrizes.
 - Complexidade cúbica.
- Problema do caixeiro viajante.
 - Complexidade exponencial.

Notação Ω (Ômega)

- A notação Ω é utilizada para definir limites inferiores assintóticos.
- Sejam f, g funções reais positivas da variável inteira n . Diz-se que $f(n)$ é $\Omega(g(n))$ quando existirem duas constantes c e m , sendo $c > 0$ e $n > m$, tais que
 - $f(n) \geq c \cdot g(n)$.
 - A partir de um inteiro m , para todo valor n superior a m , temos que $f(n)$ será sempre maior do que $c \cdot g(n)$. Então $g(n)$ será um limite assintótico inferior para a função $f(n)$.
- Exemplo:
 - Se $f(n) = 3n^3 + 2n^2$ é $\Omega(n^3)$, temos que, se $c = 1$, então $3n^3 + 2n^2 \geq n^3$ para $n \geq 0$.

Notação Θ (Teta)

- Utilizada para definir limites superiores justos.
- Sejam $f(n)$, $g(n)$ funções reais positivas da variável inteira n . Diz-se que $f(n)$ é $\Theta(g(n))$, quando ambas as condições $f(n) = O(g(n))$ e $g(n) = O(f(n))$ são verificadas.
- Outra definição:
 - Sejam $f(n)$, $g(n)$ funções reais positivas da variável inteira n . Diz-se que $f(n) = \Theta(g(n))$, se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$
- As funções possuem a mesma ordem de grandeza assintótica.
- Exemplo:
 - Se $f = n^2 - 1$ e $g = n^2$, então f é $O(g)$ e g é $O(f)$, e então f é $\Theta(g)$.
 - Se $f = n^2 - 1$ e $h = n^3$, então f é $O(h)$ mas h não é $O(f)$.

Algoritmos Ótimos

- Um algoritmo é dito ser ótimo para resolver um problema P , quando a complexidade deste algoritmo for igual ao limite inferior para a solução deste problema.
 - O custo do algoritmo é o menor custo possível!
 - Se o limite inferior para o problema P for uma função f , temos que P é $\Omega(f)$. Se o algoritmo A for igual a $\Omega(f)$, então A é um algoritmo ótimo para P .
- Estabelecer qual o limite inferior para um problema exige provas matemáticas formais. Uma função ser um limite inferior para um problema é diferente de termos um algoritmo com o menor limite inferior conhecido.

Exercícios

- Escrever as seguintes funções em notação O e justifique porque:
 - $n^3 - 1$;
 - $3n^2 - n + 4$;
 - $n^2 + 2 \log n$;
 - $3n^n + 5 * 2^n$;
 - $(n-1)^n + n^{n-1}$;
 - 302 .
- Verdadeiro ou falso - porque:
 - $2^{n+1} = O(2^n)$?
 - $2^{2n} = O(2^n)$?

Exercícios

- Considere o algoritmo a seguir que ordena uma lista de valores – ordenação por seleção. Qual a complexidade dele e porquê?

Var

a : vetor;
i, j, min : índice
x : item

Begin

for i = 1 to n-1 do
begin

Min = i;
for j = i+1 to n do
if A[j].chave < A[Min].chave then
Min = j

x = A[Min]
A[Min] = A[i]
A[i] = x

end

end